

10 Unity Project Setup Guide

Documentación normativa de Cartridge & Cloud

Proyecto

Cartridge & Cloud

Versión del proyecto

0.0.26

Autor

Blas Luis Rocha González / VRM Games

Versión documental

2.0

Estado

Baseline aprobada y consolidada

Última revisión

17 de julio de 2026

Índice

10 Unity Project Setup Guide	5
title: "Cartridge & Cloud — Unity Project Setup Guide" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-07" lang: "es-ES" document_version: "2.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE" project: "Cartridge & Cloud" platform: "PC / Steam" engine: "Unity 6.3 LTS 6000.3.18f1" render_pipeline: "URP 17.3.0" application_version_reference: "0.0.26"	6
Cartridge & Cloud — Unity Project Setup Guide	6
0. Propósito	6
0.1. Jerarquía de autoridad	6
0.2. Etiquetas de estado	7
0.3. Fotografía de la baseline	7
1. Requisitos del equipo de desarrollo	8
1.1. Sistema operativo	8
1.2. Capacidad recomendada	8
1.3. Herramientas	8
1.4. Privacidad del registro de entorno	8
2. Instalación de Unity	9
2.1. Instalar Unity Hub	9
2.2. Instalar el Editor exacto	9
2.3. Módulos	9
2.4. Validación de instalación	9
3. Obtención del repositorio	10
3.1. Clonado	10
3.2. Ruta local	10
3.3. Estado Git inicial	10
3.4. Archivos generados por IDE	10
4. Control de versiones y serialización	10
4.1. Visible Meta Files	10
4.2. Force Text	11
4.3. Finales de línea	11
4.4. .gitignore	11
4.5. Git LFS	12
5. Primera apertura	12
5.1. Selección del proyecto	12
5.2. Importación inicial	12
5.3. Orden de diagnóstico	12
5.4. Safe Mode	12
5.5. Reimportación y Git	13
6. Identidad y Player Settings	13
6.1. Valores obligatorios	13
6.2. Versión	13
6.3. Valores observados que requieren revisión	13
7. Paquetes	14
7.1. Paquetes directos aprobados	14
7.2. Paquetes retirados	14
7.3. Política de versiones	14
7.4. Paquetes opcionales	14
7.5. Paquetes built-in	15
8. Estructura real de Assets	15
8.1. Raíz propia	15
8.2. Organización por responsabilidad	15
8.3. Reglas	15
8.4. Inventario observado	16
9. Assembly Definitions	16
9.1. Principio	16

9.2. Assemblies actuales	16
9.3. Reglas de dependencia	17
9.4. Root namespaces	17
9.5. Assemblies de tests	17
10. Escenas	17
10.1. Escenas versionadas	17
10.2. GUIDs	17
10.3. Flujo de producción	18
10.4. Estado de migración observado	18
10.5. TestLab	18
11. Build Profiles	18
11.1. Profile observado	18
11.2. Reglas	18
11.3. Migración a StoreInitial	18
12. ApplicationRoot y composición	19
12.1. Ownership	19
12.2. Composition roots	19
12.3. Preloaded assets	19
13. StoreInitial	19
13.1. Principio de autoría	19
13.2. Jerarquía objetivo	19
13.3. StoreInitialSceneContext	20
13.4. Estado runtime observado	20
13.5. Arquitectura y grid	20
13.6. Puerta	20
14. Runtime Asset Registry y catálogos	20
14.1. Registry	20
14.2. Reglas	21
14.3. Datos técnicos y representativos	21
15. URP y Graphics Settings	21
15.1. Pipeline global	21
15.2. PC_RPAsset	21
15.3. Mobile_RPAsset	21
15.4. Reglas URP	22
16. Quality Settings	22
16.1. Profiles observados	22
16.2. Valores PC relevantes	22
16.3. Deuda respecto a perfiles históricos	22
16.4. Política	22
17. Iluminación, volúmenes y VFX	23
17.1. Settings	23
17.2. Reglas	23
17.3. Baked, mixed y realtime	23
18. Input System	23
18.1. Configuración	23
18.2. Estado del asset	23
18.3. Reglas	23
18.4. UI y gameplay	24
19. UI y EventSystem	24
19.1. EventSystem	24
19.2. Reglas	24
19.3. Carpetas	24
20. AI Navigation y NavMesh	24
20.1. Paquete	24
20.2. Areas observadas	24
20.3. Reglas	25
20.4. Bake y runtime	25
21. Physics y Time	25
21.1. TimeManager	25

21.2. Reglas	25
21.3. Colliders	25
22. Tags, Layers y Sorting Layers	25
22.1. Estado observado	26
22.2. Consecuencia	26
22.3. Añadir una layer	26
23. Audio	26
23.1. AudioManager observado	26
23.2. Organización	26
23.3. Importación	26
23.4. AudioCatalog	27
24. Localización	27
24.1. Paquete	27
24.2. Estado observado	27
24.3. Objetivo	27
24.4. Reglas	27
25. Importación de assets: reglas generales	27
25.1. Antes de importar	27
25.2. Importar dentro de Unity	28
25.3. Herramientas representativas	28
25.4. Nombres _LOD0	28
26. Modelos FBX y LOD	28
26.1. Escala	28
26.2. Ejes y pivotes	28
26.3. LODGroup	28
26.4. Colliders	29
27. Texturas, sprites y materiales	29
27.1. Texturas	29
27.2. Sprites/UI	29
27.3. Materiales	29
27.4. MaterialPalette	29
28. Animaciones y personajes	29
28.1. Importación	29
28.2. Runtime	30
28.3. Identidad visual	30
29. Prefabs	30
29.1. Principios	30
29.2. Prefab variants	30
29.3. Overrides	30
29.4. Entorno	30
30. ScriptableObjects y datos	31
30.1. Uso	31
30.2. IDs	31
30.3. Datos técnicos	31
30.4. Edición en Play Mode	31
31. Resources y carga de assets	31
31.1. Estado	31
31.2. Política	31
32. Código Editor	31
32.1. Ubicación	31
32.2. Reglas	32
32.3. ProjectOrganization	32
33. Configuración de tests	32
33.1. Assemblies	32
33.2. Baseline esperada	32
33.3. Primera validación	32
33.4. Fallos	32
34. Validación de escenas y catálogos	33
34.1. Smoke tests	33

34.2. Missing scripts	33
35. Validación de build mínima	33
36. Player.log y diagnóstico	33
36.1. Player Log	33
36.2. Evidencia	34
37. Limpieza segura	34
37.1. Qué puede regenerarse	34
37.2. Qué no debe borrarse	34
37.3. Orden de recuperación	34
38. Problemas habituales	35
38.1. Paquetes no resueltos	35
38.2. Materiales rosas	35
38.3. Scripts no compilan	35
38.4. Tests no aparecen	35
38.5. Build incluye tests	35
38.6. StoreInitial muestra doble tienda	35
38.7. UI mueve al jugador	36
38.8. Git muestra cientos de assets	36
39. Incorporación de una máquina nueva	36
39.1. Checklist previo	36
39.2. Checklist de clonación	36
39.3. Checklist de Unity	36
39.4. Registro	37
40. Checklist antes de un sprint	37
41. Añadir un paquete	37
41.1. Definition of Ready	37
41.2. Procedimiento	37
41.3. Rechazo	37
42. Actualizar un paquete	38
42.1. Un paquete cada vez	38
42.2. Evidencia	38
42.3. URP/Input/Navigation	38
43. Actualizar Unity	38
43.1. REQUIERE ADR	38
43.2. Prohibiciones	38
43.3. Rollback	39
44. Cambios que requieren ADR	39
45. Configuración que no debe cambiarse silenciosamente	39
46. Deuda y discrepancias actuales	39
46.1. Tabla de deuda	39
46.2. Regla	40
47. Definition of Done del setup	40
48. Criterios de aceptación	40
49. Mantenimiento	41
50. Mapa de configuración serializada	42
50.1. ProjectVersion	42
50.2. ProjectSettings.asset	42
50.3. EditorSettings.asset	42
50.4. GraphicsSettings y URP Global Settings	43
50.5. QualitySettings	43
50.6. TagManager	43
50.7. TimeManager y Physics	43
50.8. AudioManager	43
50.9. EditorBuildSettings y Build Profiles	44
51. Perfiles detallados de importación	44
51.1. Arquitectura	44
51.2. Mobiliario colocable	44
51.3. Displays	44
51.4. Productos y packaging	44

51.5. Personajes	45
51.6. Texturas de entorno	45
51.7. UI	45
51.8. Audio por categoría	45
51.9. Animaciones	46
51.10. Presets	46
52. Matriz de validación por tipo de cambio	46
52.1. Cambio pequeño no significa riesgo pequeño	46
52.2. Validación de autoría	46
52.3. Validación posterior a merge	47
53. Recuperación ante daños o pérdida de referencias	47
53.1. Principio	47
53.2. Escena dañada	47
53.3. Prefab con Missing Script	47
53.4. Material perdido	47
53.5. Catálogo roto	47
53.6. Manifest dañado	48
53.7. Library corrupta	48
53.8. Conflicto de escena o prefab	48
54. Seguridad y secretos	48
54.1. Archivos prohibidos en Git	48
54.2. Variables y configuración local	48
54.3. Assets de terceros	49
54.4. Logs y builds	49
55. Campaña de aceptación de una instalación nueva	49
55.1. Fase A — Integridad estática	49
55.2. Fase B — Importación	49
55.3. Fase C — Compilación y tests	50
55.4. Fase D — Escenas	50
55.5. Fase E — Build	50
55.6. Fase F — Repositorio	50
55.7. Resultado	50
56. Handoff del entorno	50
Anexo A. Comandos de diagnóstico	51
A.1. Git	51
A.2. Comprobación de archivos	51
Anexo D. Resumen de no negociables	51
Actualización Unity W0 - cámara y pivotes	51
Estado operativo vigente tras W0	51
Wall occlusion	52
Pivotes y GroundAnchor	52
Regla de cierre y no propagación	52

10 Unity Project Setup Guide

Metadatos normativos v2.0
document_version: 2.0
project_version: 0.0.26
last_reviewed: 2026-07-17
status: APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE
lifecycle: LIVING DOCUMENT

Nota de coherencia v1.1: las reglas de tiempo, pausa, apertura, cierre, liquidación y autosave diario se interpretan conforme a ADR-0086 y ADR-0087. Los estados de sprint o baseline anteriores se mantienen únicamente como fotografía histórica cuando una actualización posterior los sustituye.

title: "Cartridge & Cloud — Unity Project Setup Guide" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-07" lang: "es-ES" document_version: "2.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE" project: "Cartridge & Cloud" platform: "PC / Steam" engine: "Unity 6.3 LTS 6000.3.18f1" render_pipeline: "URP 17.3.0" application_version_reference: "0.0.26"

Cartridge & Cloud — Unity Project Setup Guide

0. Propósito

Esta guía define cómo instalar, clonar, abrir, validar, reparar y actualizar el proyecto Unity de *Cartridge & Cloud* de forma reproducible. Su objetivo no es describir únicamente cómo crear un proyecto nuevo, sino preservar la configuración real que permite continuar el desarrollo sin introducir diferencias silenciosas entre máquinas, escenas, paquetes, assets o builds.

La guía cubre:

- requisitos de hardware y software;
- instalación exacta del Editor y módulos;
- restauración desde Git;
- configuración de ProjectSettings y Packages;
- estructura de Assets/_Project;
- límites de assemblies;
- escenas, Build Profiles y composition roots;
- URP, calidad, iluminación, audio, input y navegación;
- importación de modelos, texturas, animaciones y audio;
- prefabs, ScriptableObjects y catálogos;
- pruebas EditMode y PlayMode;
- limpieza, reimportación y recuperación;
- incorporación de una máquina nueva;
- procedimiento de cambio de Unity o paquetes;
- decisiones que requieren ADR;
- criterios de aceptación del setup.

No sustituye:

- al TDD, que define la arquitectura;
- al estándar C#, que define el código;
- a la guía de build y versionado, que define artefactos distribuibles;
- al QA Testing Plan, que define la estrategia de verificación.

0.1. Jerarquía de autoridad

Se aplica el siguiente orden:

1. 00_Enfoque_y_Alcance.md
2. 01_Game_Design_Document.md
3. 02_Vertical_Slice_Specification.md
4. 03_Technical_Design_Document.md
5. 04_Modelo_de_Datos.md
6. 05_UX_Flow.md
7. 06_Production_Roadmap_y_Sprint_Plan.md
8. 07_QA_Testing_Plan.md
9. 08_QA_Testing_Matrix.xlsx
10. 09_CSharp_Coding_Standards.md

11. Unity Project Setup Guide v0.6
12. ProjectSettings, Packages, Assets y .asmdef reales
13. ADR-0001 y ADR-0004
14. registros de entorno y paquetes de Sprint 0
15. versiones v0.5, v0.4 y v0.3

La configuración serializada real es la verdad sobre lo que abrirá Unity. Sin embargo, cuando esa configuración contradiga un requisito vigente, no se reinterpreta como norma: se registra como **deuda o migración pendiente**.

0.2. Etiquetas de estado

Etiqueta	Significado
OBLIGATORIO	Debe estar presente para abrir, compilar o validar la baseline.
OBSERVADO	Valor encontrado en los archivos suministrados.
OBJETIVO	Configuración que debe alcanzarse antes del gate indicado.
DEUDA	Diferencia conocida entre el estado observado y el objetivo.
OPCIONAL	Puede añadirse tras justificar coste y dependencia.
PROHIBIDO	No debe hacerse sin una decisión formal.
REQUIERE ADR	Cambio estructural que necesita decisión, impacto y QA.

0.3. Fotografía de la baseline

Campo	Valor vigente
Proyecto	Cartridge & Cloud
Company	VRM Games
Editor	6000.3.18f1
Revisión del Editor	5eb53e4c07
Pipeline	URP 17.3.0
Plataforma inicial	Windows x64
Versión de aplicación	0.0.26
Application Identifier	com.vrmgames.cartridgeandcloud
Input	Input System Package
Color Space	Linear
Profile de calidad para Standalone	PC
Build Profile	Windows_Development
HISTORICAL EVIDENCE — infraestructura automatizada retirada:	Pruebas documentadas
Estado	Sprints 0-15 cerrados; Sprint 16 COMPLET

Los resultados de pruebas son una referencia de la working copy de la baseline v0.6. Una máquina nueva debe volver a ejecutarlos; no debe copiar su estado como si fuera evidencia propia.

0.2.1. Cierre operativo de Sprint 16 HISTORICAL EVIDENCE — las pruebas automatizadas existentes en esa ejecución finalizaron en PASS; la infraestructura no está presente actualmente y se reimplantará en una fase futura.

1. Requisitos del equipo de desarrollo

1.1. Sistema operativo

El entorno validado utiliza Windows. La plataforma de build inicial es Windows x64.

La máquina debe disponer de:

- una versión de Windows todavía soportada por el Editor seleccionado;
- permisos para instalar Unity Hub, el Editor y módulos;
- rutas locales con permisos de lectura y escritura;
- espacio suficiente para proyecto, Library, builds y cachés;
- controlador gráfico estable;
- reloj del sistema correcto, para no distorsionar archivos, logs o certificados.

La evidencia histórica se obtuvo en Windows 10 Pro 19045 con un i9-9900K, 31,9 GB de RAM y RTX 3060. Ese hardware no es un requisito mínimo oficial; es el entorno de referencia que debe registrarse al comparar rendimiento.

1.2. Capacidad recomendada

Para trabajo cómodo:

- 16 GB de RAM como mínimo práctico; 32 GB recomendado;
- SSD con espacio libre amplio;
- GPU compatible con las capacidades requeridas por Unity 6 y URP;
- CPU multinúcleo moderna;
- monitor que permita validar al menos 1920×1080;
- segundo espacio de almacenamiento para artefactos o backups cuando sea posible.

No se debe inferir un requisito comercial de hardware a partir de estas recomendaciones. Los requisitos de Steam se fijarán después del profiling de Sprint 17.

1.3. Herramientas

OBLIGATORIO:

- Unity Hub;
- Unity 6000.3.18f1;
- Windows Build Support para esa instalación;
- IDE C# compatible;
- Git;
- acceso al repositorio;
- herramienta para revisar cambios Git.

El entorno histórico usa Visual Studio Community y GitHub Desktop. El repositorio contiene .vs-config con el workload Microsoft.VisualStudio.Workload.ManagedGame.

1.4. Privacidad del registro de entorno

Los registros pueden incluir:

- versión de Windows;
- CPU, RAM, GPU y driver;
- versión de Unity;
- versión del IDE;
- volumen de disco y espacio aproximado;
- versión, SHA y resultados.

No deben incluir:

- usuario del sistema;
 - nombre del equipo;
 - tokens;
 - rutas personales completas;
 - seriales;
 - credenciales;
 - claves de firma;
 - correo privado no necesario.
-

2. Instalación de Unity

2.1. Instalar Unity Hub

1. Instalar Unity Hub desde un instalador confiable.
2. Iniciar sesión solo si la licencia lo requiere.
3. Activar una licencia válida.
4. Confirmar que Hub puede escribir en la ubicación de Editors y Projects.
5. No crear todavía un proyecto nuevo para reemplazar el repositorio.

2.2. Instalar el Editor exacto

Instalar **Unity 6.3 LTS 6000.3.18f1**.

No utilizar:

- otra revisión 6000.3.x por considerarla equivalente;
- Unity 6.0 o 6.1;
- una Tech Stream;
- una beta;
- una versión más nueva “compatible”.

ProjectSettings/ProjectVersion.txt contiene:

```
m_EditorVersion: 6000.3.18f1  
m_EditorVersionWithRevision: 6000.3.18f1 (5eb53e4c07)
```

Una actualización del Editor requiere el procedimiento del capítulo 45.

2.3. Módulos

Instalar:

- Windows Build Support;
- Microsoft Visual Studio Community o integración con el IDE elegido;
- documentación local solo si aporta valor;
- símbolos o herramientas de profiling cuando la campaña los requiera.

No instalar módulos móviles o de consola como supuesto compromiso de plataforma.

2.4. Validación de instalación

Antes de abrir el proyecto:

- verificar la versión en Hub;
- verificar Windows Build Support;
- confirmar licencia;
- comprobar espacio libre;
- registrar versión del IDE;
- comprobar que Git funciona.

3. Obtención del repositorio

3.1. Clonado

Clonar el repositorio completo, no descargar una carpeta parcial de Assets.

Deben estar presentes en la raíz:

```
Assets/  
Packages/  
ProjectSettings/  
Documentation/  
Tools/  
.gitignore  
.gitattributes  
.vsconfig
```

Library, Temp, Obj, Logs, UserSettings y builds locales se generan o permanecen locales.

3.2. Ruta local

La ruta debe:

- ser local;
- evitar sincronización en tiempo real de servicios cloud;
- evitar permisos restringidos;
- evitar rutas extremadamente largas;
- disponer de espacio;
- no estar dentro de otra working copy Unity.

No abrir el proyecto desde un .zip ni desde una carpeta temporal.

3.3. Estado Git inicial

Antes de abrir:

1. comprobar rama;
2. hacer fetch/pull;
3. confirmar que no hay cambios locales;
4. registrar SHA;
5. verificar que no faltan archivos LFS, si en el futuro se introduce LFS;
6. no generar .meta manualmente.

3.4. Archivos generados por IDE

Los .csproj, .sln y .slnx generados por Unity están ignorados. El paquete recibido puede contener copias para auditoría, pero la working copy normal debe regenerarlos desde Unity.

No corregir una referencia de assembly editando un .csproj generado.

4. Control de versiones y serialización

4.1. Visible Meta Files

VersionControlSettings.asset está configurado en Visible Meta Files.

Reglas:

- todo asset versionado conserva su .meta;
- mover assets se realiza dentro de Unity cuando sea posible;
- si se mueve fuera de Unity, se mueve también el .meta;
- no borrar .meta para “forzar” una reparación;
- no copiar un asset con el .meta original si se necesita un GUID nuevo;
- no regenerar el .meta de un prefab referenciado.

4.2. Force Text

EditorSettings.asset contiene m_SerializationMode: 2, correspondiente a la política de serialización textual de la baseline.

Esto permite revisar y fusionar:

- escenas;
- prefabs;
- assets;
- materiales;
- animaciones;
- configuraciones.

Los conflictos no deben resolverse aceptando automáticamente “ours” o “theirs” sin comprender las referencias.

4.3. Finales de línea

El repositorio normaliza a LF mediante .gitattributes:

- C#;
- JSON;
- Markdown;
- YAML;
- .meta;
- escenas;
- prefabs;
- assets;
- shaders y UI.

Los binarios se marcan como binary. La configuración serializada del Editor para nuevos scripts no sustituye a .gitattributes.

4.4. .gitignore

Se ignoran, entre otros:

```
Library/  
Temp/  
Obj/  
Build/  
Builds/  
Logs/  
UserSettings/  
MemoryCaptures/  
Recordings/  
.vs/  
.idea/  
*.csproj  
*.sln  
*.slnx
```

Se versionan:

- Assets con .meta;
- Packages;
- ProjectSettings;
- documentación;
- herramientas;
- evidencia explícitamente aceptada.

4.5. Git LFS

OBSERVADO: no existe una política LFS activa en la .gitattributes suministrada. Los binarios se clasifican como binarios normales.

OBJETIVO: introducir LFS solo cuando el crecimiento real de FBX, PSD, WAV o vídeo lo justifique y exista un plan de migración. No activar patrones LFS parcialmente en mitad de un sprint sin verificar clones limpios.

5. Primera apertura

5.1. Selección del proyecto

En Unity Hub:

1. elegir Open;
2. seleccionar la raíz que contiene Assets, Packages y ProjectSettings;
3. confirmar el Editor exacto;
4. aceptar la reimportación;
5. no seleccionar una subcarpeta.

5.2. Importación inicial

La primera apertura puede regenerar Library y tardar significativamente.

Durante la importación:

- no cerrar el Editor salvo bloqueo real;
- no mover assets;
- no ejecutar herramientas de autoría;
- no modificar Package Manager;
- no interpretar warnings transitorios como estado final.

5.3. Orden de diagnóstico

Después de la importación:

1. esperar que desaparezca la actividad;
2. revisar Console;
3. resolver errores rojos desde el primero;
4. comprobar Package Manager;
5. comprobar assemblies;
6. comprobar escenas;
7. ejecutar tests;
8. comprobar Git.

5.4. Safe Mode

Si Unity entra en Safe Mode:

- leer el primer error de compilación;
- corregir referencia, paquete o archivo;
- no borrar assets al azar;
- no reimportar todo como primer paso;
- no cambiar asmdefs para ocultar el problema;
- no salir de Safe Mode sin una compilación coherente.

5.5. Reimportación y Git

Tras una apertura limpia, Git no debe mostrar cambios inesperados. Cambios masivos en .meta, escenas o materiales indican:

- Editor incorrecto;
- import settings diferentes;
- archivos faltantes;
- generación automática no idempotente;
- normalización no aplicada.

Detenerse antes de hacer commit.

6. Identidad y Player Settings

6.1. Valores obligatorios

Campo	Valor
Company Name	VRM Games
Product Name	Cartridge & Cloud
Identifier Standalone	com.vrmgames.cartridgeandcloud
Bundle Version	0.0.26 en la baseline vigente
Color Space	Linear
Player Log	Activado
Input Handling	Input System Package

6.2. Versión

No incrementar bundleVersion por cada apertura. Se cambia cuando el sprint o build lo exige y se registra junto con SHA, schema y evidencia.

6.3. Valores observados que requieren revisión

La configuración suministrada contiene:

Ajuste	Valor observado	Tratamiento
resolución por defecto	1024 × 768	DEUDA: decidir antes de H6; QA usa 1920×1080 y otras res
ventana redimensionable	desactivada	validar decisión UX antes de release
run in background	desactivado	mantener salvo requisito explícito
fullscreen mode serializado	1	validar comportamiento en build, no inferir solo del YAML
VSync en calidad PC	0	el frame cap debe definirse y probarse
Player Log	activado	obligatorio en builds internas
Flip Model Swapchain	activado	validar en hardware objetivo
GPU Skinning	activado	medir con personajes representativos

Ajuste	Valor observado	Tratamiento
deterministic compilation	activado	conservar
unsafe code	desactivado	conservar salvo ADR

No cambiar varios de estos valores simultáneamente sin una build comparativa.

7. Paquetes

7.1. Paquetes directos aprobados

Paquete	Versión	Uso
com.unity.ai.navigation	2.0.13	navegación y NavMesh
com.unity.ide.visualstudio	2.0.26	integración IDE
com.unity.inputsystem	1.19.0	input
com.unity.localization	1.5.12	localización
com.unity.render-pipelines.universal	17.3.0	render pipeline
com.unity.test-framework	1.6.0	EditMode/PlayMode
com.unity.ugui	2.0.0	UI

manifest.json y packages-lock.json se versionan juntos.

7.2. Paquetes retirados

El registro de Sprint 0 retiró:

- Rider Editor;
- Collab Proxy;
- Multiplayer Center;
- Visual Scripting;
- Timeline.

No reintroducirlos por plantilla o conveniencia sin uso real.

7.3. Política de versiones

- solo paquetes Released;
- no usar Preview;
- no editar packages-lock.json manualmente;
- no actualizar todos los paquetes a la vez;
- no aceptar una actualización automática sin diff;
- comprobar changelog y compatibilidad;
- ejecutar suites y build tras cada cambio.

7.4. Paquetes opcionales

Paquete/capacidad	Condición
Cinemachine	spike que demuestre valor frente a la cámara actual
Addressables	volumen de contenido y estrategia de distribución justificados

Paquete/capacidad	Condición
Steamworks	build representativa y plan de integración aprobado
Git LFS	crecimiento del repositorio y migración aprobados
analyzers	plan gradual del estándar C#

7.5. Paquetes built-in

El manifiesto incluye módulos built-in de Unity. No se interpretan como plataformas comprometidas. No retirar módulos individuales a mano para “limpiar” el manifiesto sin comprobar sus dependencias.

8. Estructura real de Assets

8.1. Raíz propia

Todo contenido propio vive bajo Assets/_Project.

```
Assets/_Project/  
├── Animations/  
├── Art/  
├── Audio/  
├── Content/  
├── Data/  
├── Editor/  
├── Prefabs/  
├── Scenes/  
├── Scripts/  
├── Settings/  
├── Tests/  
├── UI/  
└── InputSystem_Actions.inputactions
```

8.2. Organización por responsabilidad

- Art: modelos, texturas, materiales, sprites y VFX.
- Audio: ambiente, music, SFX y UI.
- Content: localización y contenido de autoría no técnico.
- Data: catálogos y definiciones ScriptableObject.
- Editor: herramientas que solo compilan en Editor.
- Prefabs: arquitectura, muebles, productos, personajes y expansiones.
- Scenes: escenas versionadas.
- Scripts: capas y runtime.
- Settings: URP, Build Profiles y registros runtime.
- Tests: suites EditMode, PlayMode e Input System.
- UI: fuentes, iconos, estilos y vistas.

8.3. Reglas

- no crear carpetas genéricas Misc, New Folder o Shared para código;
- Generated solo para assets cuyo origen y regeneración estén documentados;
- separar assets técnicos de representativos;
- no guardar evidencia o builds dentro de Assets;
- no colocar scripts fuera de un assembly propio;
- no usar Resources como repositorio general.

8.4. Inventario observado

La baseline contiene aproximadamente:

Área	Archivos sin .meta
Animations	11
Art	140
Audio	12
Content	3
Data	42
Editor	16
Prefabs	68
Scenes	5
Scripts	306
Settings	10
Tests	147
UI	4

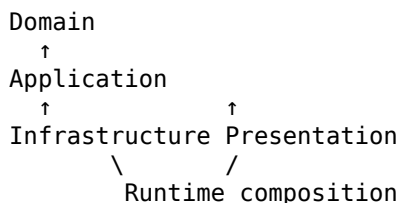
Los conteos son una fotografía, no un objetivo. El gate valida integridad y función, no cantidad.

9. Assembly Definitions

9.1. Principio

Los assemblies reducen acoplamiento y tiempo de compilación, y protegen la simulación de referencias indebidas a Unity.

Grafo principal:



9.2. Assemblies actuales

La baseline ya no contiene solo los seis assemblies de Sprint 0; existen doce:

Assembly	Referencias directas
VRMGames.CartridgeAndCloud.Editor.ProjectOrganization	VRMGames.CartridgeAndCloud.Domain,
VRMGames.CartridgeAndCloud.Editor	VRMGames.CartridgeAndCloud.Applicati
VRMGames.CartridgeAndCloud.Application	VRMGames.CartridgeAndCloud.Domain
VRMGames.CartridgeAndCloud.Domain	Ninguna
VRMGames.CartridgeAndCloud.Infrastructure.InputSystem	VRMGames.CartridgeAndCloud.Applicati
VRMGames.CartridgeAndCloud.Infrastructure	VRMGames.CartridgeAndCloud.Domain,
VRMGames.CartridgeAndCloud.Presentation	VRMGames.CartridgeAndCloud.Domain,
VRMGames.CartridgeAndCloud.Runtime.VerticalSlicePhase1	VRMGames.CartridgeAndCloud.Domain,
VRMGames.CartridgeAndCloud.Tests.EditMode	VRMGames.CartridgeAndCloud.Domain,
VRMGames.CartridgeAndCloud.InputSystem.Tests.EditMode	VRMGames.CartridgeAndCloud.Applicati

Assembly	Referencias directas
<code>VRMGames.CartridgeAndCloud.InputSystem.Tests.PlayMode</code>	<code>VRMGames.CartridgeAndCloud.Application</code>
<code>VRMGames.CartridgeAndCloud.Tests.PlayMode</code>	<code>VRMGames.CartridgeAndCloud.Domain</code>

9.3. Reglas de dependencia

- Domain no referencia Unity ni otros assemblies propios.
- Application referencia Domain y no Unity.
- Infrastructure implementa puertos y referencia Domain/Application.
- Presentation referencia contratos, no implementaciones de Infrastructure.
- el assembly `InputSystem` encapsula el paquete de input;
- runtime compone dependencias explícitas;
- tests no entran en Player salvo constraints aprobadas;
- código Editor usa `includePlatforms: Editor`;
- nuevas referencias requieren revisión.

9.4. Root namespaces

Los `asmdefs` establecen root namespaces explícitos. `EditorSettings` no define un root namespace global. No rellenarlo sin comprobar el impacto sobre plantillas y namespaces generados.

9.5. Assemblies de tests

Los `PlayMode` usan `UNITY_INCLUDE_TESTS` donde se necesita impedir compilación accidental en el Player. Un fallo histórico de build demostró que esta restricción es obligatoria.

10. Escenas

10.1. Escenas versionadas

Escena	Función	Estado
<code>Bootstrap.unity</code>	entrada y <code>ApplicationRoot</code>	producción
<code>MainMenu.unity</code>	menú y slots	producción
<code>Store.unity</code>	escena funcional anterior/fallback	migración
<code>StoreInitial.unity</code>	tienda autorada objetivo	Sprint 16
<code>TestLab.unity</code>	pruebas aisladas	desarrollo

10.2. GUIDs

Los GUIDs observados confirman que `StoreInitial` es un asset distinto:

Escena	GUID
<code>Bootstrap</code>	<code>ada1e52ac6e09b9468e2109ccc7cbfc0</code>
<code>MainMenu</code>	<code>b00786c6952a55d418cf16503030c766</code>
<code>Store</code>	<code>45adcce906944f74bad3a0fb7d62f8e5</code>
<code>StoreInitial</code>	<code>32a95e5e16aaf15449a6f81392cf5dcf</code>
<code>TestLab</code>	<code>002b2ad62af3c614b91e32722c9452be</code>

10.3. Flujo de producción

Bootstrap → MainMenu → Slot Flow → StoreInitial

Bootstrap permanece primera. No entrar directamente en MainMenu o StoreInitial para validar producción, salvo un test aislado que lo declare.

10.4. Estado de migración observado

La configuración validada para el cierre de Sprint 16 utiliza el flujo:

Bootstrap
MainMenu
StoreInitial

StoreInitial está conectada al runtime y fue validada mediante build Windows x64 externa 0.0.26, Golden Path, persistencia y revisión de Player.log. Store permanece como escena histórica y TestLab no forma parte del recorrido productivo de cierre.

PROHIBIDO: degradar esta configuración, reintroducir Store como destino productivo o incluir TestLab en una candidata H6 sin una decisión y validación explícitas.

10.5. TestLab

TestLab puede permanecer en un profile interno. La futura build candidata de hito debe decidir si lo excluye. Nunca debe aparecer como opción de usuario.

11. Build Profiles

11.1. Profile observado

Assets/_Project/Settings/BuildProfiles/Windows_Development.asset:

- target Windows Standalone;
- desarrollo activado;
- compresión configurada;
- lista de escenas propia;
- profiler no conectado por defecto;
- deep profiling desactivado;
- debugging gestionado desactivado.

11.2. Reglas

- versionar el Build Profile;
- no depender de la lista global si el profile la sobrescribe;
- validar que Bootstrap es índice 0;
- no editar YAML a mano salvo recuperación excepcional;
- generar el profile desde Unity;
- registrar cambios de escenas;
- separar Development, QA, Candidate y Release cuando proceda.

11.3. Migración a StoreInitial

Cuando Sprint 16 cumpla sus pasos:

1. validar escena en Editor;
2. validar context;
3. desactivar blackout procedural en la ruta nueva;
4. ejecutar suites;

5. sustituir Store por StoreInitial en el profile;
 6. generar build;
 7. ejecutar EXE;
 8. revisar Player.log;
 9. conservar fallback hasta aceptar la migración;
 10. registrar decisión de retirada.
-

12. ApplicationRoot y composición

12.1. Ownership

ApplicationRoot pertenece a Bootstrap y persiste durante transiciones. No crear copias en Main-Menu, Store o StoreInitial.

12.2. Composition roots

La composición debe:

- crear servicios una vez;
- inyectar contratos;
- registrar escenas;
- separar runtime de objetos visuales;
- evitar service locator global;
- fallar con diagnóstico si falta una referencia.

12.3. Preloaded assets

ProjectSettings.asset referencia como preloaded assets:

- InputSystem_Actions.inputactions;
- RuntimeAssetRegistry.asset.

Eliminar o sustituir estas entradas puede romper bootstrap y catálogos. Requiere tests de arranque, input y registry.

13. StoreInitial

13.1. Principio de autoría

StoreInitial.unity se autoriza manualmente. Runtime registra y opera el entorno; no reconstruye la arquitectura desde nombres, bounds o jerarquías de FBX.

13.2. Jerarquía objetivo

```
StoreInitialEnvironment
├── Architecture
├── InitialFurniture
├── Lighting
├── Anchors
└── TechnicalColliders
```

13.3. StoreInitialSceneContext

Debe serializar referencias explícitas a:

- environment root;
- entrada y salida;
- recepción;
- checkout;
- puntos de clientes;
- roots de muebles y displays;
- colliders técnicos;
- iluminación o volumen necesarios;
- elementos de puerta.

Debe validar referencias antes de devolver control.

13.4. Estado runtime observado

StoreRuntimeSettings.asset todavía contiene:

```
_storeSceneName: Store  
_buildBlockoutOnLoad: 1  
_hideOccludingWalls: 1  
_showProcedurePanel: 1  
_vfxPoolSize: 24  
_maximumBlockoutCustomers: 4
```

Esto es una **DEUDA DE MIGRACIÓN**. No cambiar `_storeSceneName` y `blockout` hasta que la escena nueva esté conectada y validada.

13.5. Arquitectura y grid

- tamaño aproximado: 10 × 15 m;
- grid lógico: 20 × 30;
- celda: 0,5 m;
- colliders y grid son autoridad técnica;
- visual y colisión no deben contradecirse;
- entrada, checkout, almacén y recepción deben ser alcanzables.

13.6. Puerta

La puerta debe usar referencias explícitas. No deducir hojas por nombre. Validar:

- posición cerrada;
- dirección de apertura;
- colliders;
- trigger;
- navegación;
- estado de jornada;
- carga/restauración si aplica.

14. Runtime Asset Registry y catálogos

14.1. Registry

RuntimeAssetRegistry.asset centraliza referencias a:

- settings;

- content catalog;
- store shell;
- material palette;
- presentation catalog;
- audio catalog;
- representative prefab catalog.

14.2. Reglas

- no buscar catálogos con `Resources.FindObjectsOfTypeAll`;
- no duplicar registries por escena;
- mantener IDs estables;
- validar referencias nulas en Editor y runtime;
- no crear assets de catálogo en Play Mode;
- no reordenar datos si el orden tiene semántica sin migración.

14.3. Datos técnicos y representativos

Las carpetas `Data/*/Technical` contienen assets válidos para integración y pruebas. No deben confundirse con contenido comercial final. Un catálogo representativo sustituye referencias, no invariantes de dominio.

15. URP y Graphics Settings

15.1. Pipeline global

`GraphicsSettings.asset` apunta a `PC_RPAsset.asset`.

Valores observados:

- luces con intensidad lineal;
- temperatura de color activada;
- pipeline PC global;
- SRP Batcher activado en assets URP.

15.2. PC_RPAsset

Ajuste	Valor observado
HDR	activado
Render Scale	1.0
sombras de luz principal	activadas
sombras de luces adicionales	activadas
distancia de sombras	50
cascadas	4
SRP Batcher	activado

15.3. Mobile_RPAsset

Existe como configuración secundaria:

- Render Scale 0.8;
- una cascada;
- sombras adicionales desactivadas;

- SRP Batcher activado.

Su existencia no compromete una versión móvil.

15.4. Reglas URP

- no cambiar pipeline asset desde una escena;
 - no duplicar materiales por error de shader;
 - no convertir todos los materiales automáticamente sin revisión;
 - no activar features caras sin profiling;
 - probar profundidad, sombras y transparencias en build;
 - mantener materiales compatibles con URP;
 - registrar renderer features.
-

16. Quality Settings

16.1. Profiles observados

Existen dos perfiles:

- Mobile;
- PC.

Standalone usa PC y el índice actual es el profile PC.

16.2. Valores PC relevantes

- pixel lights: 2;
- sombras activadas;
- shadow distance: 40 en Quality Settings;
- dos cascadas en Quality Settings;
- MSAA/antiAliasing: 0 en Quality Settings;
- VSync: 0;
- LOD bias: 2;
- anisotropic textures: 2;
- mip streaming desactivado.

URP contiene ajustes adicionales; el comportamiento efectivo se valida visualmente y con Profiler.

16.3. Deuda respecto a perfiles históricos

La guía v0.3 proponía Low/Medium/High, pero la baseline real usa Mobile/PC. No crear tres perfiles por cumplir un documento antiguo. Primero definir necesidades de usuario y coste de mantenerlos.

16.4. Política

- cambiar un ajuste por hipótesis medible;
 - registrar comparación antes/después;
 - no optimizar sin profiler;
 - validar en build;
 - no usar calidad para ocultar assets mal importados;
 - revisar sombras en StoreInitial.
-

17. Iluminación, volúmenes y VFX

17.1. Settings

La carpeta contiene:

- DefaultVolumeProfile.asset;
- SampleSceneProfile.asset;
- renderer assets;
- pipeline global.

17.2. Reglas

- un volumen global debe tener ownership claro;
- evitar múltiples volúmenes globales no intencionados;
- no usar postprocesado para compensar iluminación defectuosa;
- conservar legibilidad de UI;
- revisar exposición en entrada, displays, almacén y checkout;
- validar parpadeos y flashes;
- VFX críticos tienen alternativa visual o textual.

17.3. Baked, mixed y realtime

La elección debe basarse en:

- movilidad de luces;
- autoría de StoreInitial;
- coste de bake;
- iteración;
- memoria;
- rendimiento.

No iniciar un bake grande justo antes de un commit sin asegurar que los datos generados están correctamente versionados o excluidos según política.

18. Input System

18.1. Configuración

El paquete 1.19.0 está instalado y `activeInputHandler` utiliza la ruta del Input System. `Input-System_Actions.inputactions` está preloaded.

18.2. Estado del asset

El asset contiene mapas y bindings generados por la plantilla, incluidos controles que no todos corresponden al diseño final. La arquitectura real usa contextos y adaptadores propios.

DEUDA: revisar acciones genéricas como Attack, Crouch o Jump antes de considerarlas contrato del juego.

18.3. Reglas

- no leer `Keyboard.current` desde Domain/Application;
- encapsular Input System en Infrastructure;
- Presentation consume contratos;
- UI tiene prioridad;
- bindings visibles se obtienen del sistema;

- no codificar iconos o nombres de teclas como texto fijo;
- tests de Input System permanecen en assemblies propios.

18.4. UI y gameplay

Antes de raycast de mundo, comprobar UI. Los contextos deben ser exclusivos:

None
MainMenuUI
Exploration
Interaction
BuildMode
ModalUI
Paused
Results

Abrir o cerrar UI no debe mover, colocar, retirar ni cobrar.

19. UI y EventSystem

19.1. EventSystem

Cada escena operativa debe tener exactamente el EventSystem previsto por composición. No crear uno en prefabs de entorno.

19.2. Reglas

- una UI modal captura input;
- el clic que cierra un panel no se propaga;
- Escape resuelve el nivel más profundo;
- el foco vuelve al control origen;
- los botones destructivos exigen confirmación;
- Canvas y escalado se validan en 1280×720, 1920×1080 y 2560×1440;
- no depender solo del color.

19.3. Carpetas

UI/Fonts, UI/Icons, UI/Styles y UI/Views existen como estructura. Mantener assets visuales separados de lógica.

20. AI Navigation y NavMesh

20.1. Paquete

AI Navigation 2.0.13 es dependencia directa.

20.2. Áreas observadas

Solo existen las áreas estándar:

- Walkable;
- Not Walkable;
- Jump.

No hay áreas propias registradas.

20.3. Reglas

- no crear un área por cada tipo de mueble;
- no cambiar costes sin caso de uso;
- placement debe invalidar o actualizar acceso según arquitectura;
- entrada, checkout y salida son puntos obligatorios;
- validar ruta antes de abrir tienda;
- un cliente bloqueado no impide cierre;
- no asumir que un collider visual es automáticamente navegación.

20.4. Bake y runtime

Documentar:

- surface;
- agent type;
- layers incluidas;
- volumen;
- overrides;
- método de actualización.

Un cambio de escala o puerta exige rebake o actualización coherente.

21. Physics y Time

21.1. TimeManager

Valores observados:

- Fixed Timestep: 0.02;
- Maximum Allowed Timestep: 0.33333334;
- Time Scale: 1.

21.2. Reglas

- no usar FixedUpdate para lógica que no dependa de física;
- no cambiar timestep para corregir rendimiento;
- el tiempo de jornada usa un reloj lógico propio;
- pause no deja transacciones parciales;
- pruebas deterministas controlan el tiempo;
- no usar tiempo real local para simular días.

21.3. Colliders

- ajustar al volumen jugable;
 - evitar colliders invisibles engañosos;
 - usar colliders simples cuando sea posible;
 - no usar MeshCollider convexo indiscriminadamente;
 - validar puertas y pasillos;
 - distinguir trigger y bloqueo físico.
-

22. Tags, Layers y Sorting Layers

22.1. Estado observado

- no existen tags personalizados;
- layers personalizadas no están definidas;
- Sorting Layer: Default;
- Rendering Layers: Default y Light Layer 1–7.

22.2. Consecuencia

El proyecto no debe depender de strings de tags inexistentes. La selección y composición se resuelven mediante referencias, componentes y máscaras serializadas.

22.3. Añadir una layer

Antes de añadir:

1. definir uso;
2. comprobar si un componente basta;
3. asignar índice y nombre;
4. actualizar collision matrix;
5. actualizar cámaras y raycasts;
6. actualizar pruebas;
7. documentar.

Una layer no se renombra después de usarse sin auditar assets serializados.

23. Audio

23.1. AudioManager observado

Ajuste	Valor
volumen global	1
speaker mode	serializado 2
sample rate	sistema (0)
DSP buffer	1024
virtual voices	512
real voices	32

23.2. Organización

Audio/
├─ Ambience
├─ Music
├─ SFX
└─ UI

23.3. Importación

- WAV fuente cuando proceda;
- compresión según duración y uso;
- Decompress On Load para clips breves críticos cuando esté justificado;
- Streaming para música larga tras medir;
- normalizar sin destruir dinámica;
- evitar clipping;

- loop points revisados;
- no duplicar el mismo clip por escena.

23.4. AudioCatalog

Los clips runtime se registran en catálogo. No usar rutas mágicas. Las alertas importantes disponen de alternativa visual.

24. Localización

24.1. Paquete

Localization 1.5.12 está instalado.

24.2. Estado observado

Content/Localization contiene únicamente un placeholder `.gitkeep` en el paquete suministrado. Esto significa que la infraestructura está preparada, pero no demuestra que tablas ES/EN estén cerradas.

24.3. Objetivo

Antes de H6:

- definir locales ES/EN;
- crear String Tables;
- usar claves estables;
- eliminar textos visibles hardcoded;
- probar plurales y parámetros;
- probar formatos de moneda/fecha;
- validar expansión y truncamiento;
- persistir idioma.

24.4. Reglas

- no usar texto español como clave;
 - no concatenar frases localizadas;
 - no almacenar referencias a tablas por nombre variable sin validación;
 - no borrar claves usadas sin migración;
 - exportaciones de traducción deben conservar IDs.
-

25. Importación de assets: reglas generales

25.1. Antes de importar

Comprobar:

- licencia y procedencia;
- escala;
- ejes;
- unidades;
- pivote;
- nombres;
- materiales;

- texturas;
- animaciones;
- LODs;
- colliders;
- destino en carpetas.

25.2. Importar dentro de Unity

- copiar al destino final o a staging documentado;
- esperar importación;
- revisar Inspector;
- crear prefab;
- probar en escena aislada;
- registrar en catálogo si procede;
- no referenciar directamente el FBX desde todos los sistemas.

25.3. Herramientas representativas

La herramienta de importación representativa se ejecuta solo cuando cambian FBX o reglas de importación. Debe ser idempotente.

No ejecutar herramientas masivas sobre una working copy sucia.

25.4. Nombres _LOD0

Los archivos principales pueden terminar en _LOD0 y contener LOD0/1/2 internos. No interpretar el sufijo como ausencia de otros niveles sin inspeccionar el asset.

26. Modelos FBX y LOD

26.1. Escala

- 1 unidad Unity equivale a 1 metro;
- validar tamaño con grid de 0,5 m;
- no corregir escala solo en el Transform de cada instancia;
- preferir import scale coherente;
- documentar excepciones.

26.2. Ejes y pivotes

- forward/up coherentes;
- pivote adecuado para placement;
- muebles apoyan en suelo;
- puertas pivotan según diseño;
- productos se alinean con slots.

26.3. LODGroup

- transferir LODs al prefab, no deducirlos en runtime;
- revisar thresholds en cámara real;
- evitar popping visible;
- LOD final puede usar simplificación o culling;
- colliders no cambian de forma impredecible con LOD;
- medir coste total.

26.4. Colliders

- arquitectura usa colliders técnicos explícitos;
 - muebles colocables definen footprint y colisión;
 - evitar MeshCollider de alta densidad;
 - el collider no se deriva por bounds para arquitectura autorada.
-

27. Texturas, sprites y materiales

27.1. Texturas

Configurar:

- texture type;
- sRGB según naturaleza;
- normal map correctamente;
- max size;
- compresión;
- mipmaps;
- alpha;
- wrap/filter.

No usar la misma importación para UI, albedo, normal y máscaras.

27.2. Sprites/UI

- Sprite type apropiado;
- pivote consistente;
- pixels per unit documentado;
- atlas solo cuando exista estrategia;
- iconos legibles a escala real;
- evitar texto incrustado.

27.3. Materiales

- URP shaders;
- paleta registrada;
- no duplicar material por instancia;
- no modificar `sharedMaterial` en runtime sin intención;
- usar `MaterialPropertyBlock` para variaciones apropiadas;
- validar batching y memoria.

27.4. MaterialPalette

El catálogo de materiales actúa como fuente de referencias representativas. No usar búsquedas por nombre de material.

28. Animaciones y personajes

28.1. Importación

- rig correcto;
- avatar validado;
- clips nombrados;

- loop revisado;
- root motion decidido;
- eventos de animación mínimos y documentados;
- compresión comparada visualmente.

28.2. Runtime

La animación no es autoridad del dominio. Un evento visual no debe confirmar una venta o transferencia.

28.3. Identidad visual

- dependientes mantienen cohesión de VRM Games;
 - clientes y proveedores pueden variar;
 - perfiles lógicos no se identifican por nombre del prefab;
 - los prefabs se registran mediante catálogo.
-

29. Prefabs

29.1. Principios

Un prefab debe tener:

- responsabilidad clara;
- nombre estable;
- componentes requeridos;
- referencias serializadas;
- collider/renderers coherentes;
- ausencia de managers globales;
- prueba o validación de autoría cuando sea crítico.

29.2. Prefab variants

Usar variants cuando comparten estructura y el vínculo aporta valor. Evitar cadenas profundas de variantes que oculten overrides.

29.3. Overrides

Antes de aplicar overrides:

- revisar cada cambio;
- no aplicar transform de una instancia de escena accidentalmente;
- no aplicar referencias a objetos de escena;
- comprobar impacto en otras escenas.

29.4. Entorno

`StoreInitialEnvironment.prefab` contiene entorno fijo. No incluye `EventSystem`, HUD, cámara, servicios, save ni `ApplicationRoot`.

30. ScriptableObjects y datos

30.1. Uso

ScriptableObjects son apropiados para:

- definiciones;
- catálogos;
- configuración;
- paletas;
- registry;
- settings de autoría.

No son el estado mutable autoritativo de una partida.

30.2. IDs

- IDs estables;
- no usar nombre de asset como identidad persistente;
- no regenerar IDs al reimportar;
- validar duplicados;
- conservar referencias durante rename/move.

30.3. Datos técnicos

Los assets técnicos permiten pruebas e integración. Deben estar claramente marcados y no mezclarse con contenido representativo sin decisión.

30.4. Edición en Play Mode

Los cambios accidentales sobre ScriptableObjects pueden persistir. Las herramientas y tests deben clonar o restaurar cuando proceda.

31. Resources y carga de assets

31.1. Estado

Existe Assets/_Project/Resources, pero no debe convertirse en ruta general de carga.

31.2. Política

Usar referencias serializadas, registries o catálogos. Resources.Load solo con justificación, ruta estable y test.

No introducir Addressables hasta que se apruebe su coste y migración.

32. Código Editor

32.1. Ubicación

Herramientas Editor viven en:

- Assets/_Project/Editor;
- assemblies Editor-only.

32.2. Reglas

- usar Undo;
- marcar dirty correctamente;
- guardar assets explícitamente;
- preservar .meta;
- ser idempotente;
- mostrar resumen antes de cambios masivos;
- soportar dry-run cuando el riesgo lo justifique;
- no ejecutar automáticamente al abrir proyecto sin motivo crítico.

32.3. ProjectOrganization

Las herramientas de organización pueden crear, validar o reparar estructura. No deben mover assets representativos silenciosamente ni alterar GUIDs.

33. Configuración de tests

33.1. Assemblies

Existen suites:

- EditMode general;
- PlayMode general;
- EditMode de Input System;
- PlayMode de Input System.

33.2. Baseline esperada

La working copy documentada reporta:

HISTORICAL EVIDENCE — infraestructura automatizada retirada: - 1215 EditMode PASS; HISTORICAL EVIDENCE — infraestructura automatizada retirada: - 70 PlayMode PASS; HISTORICAL EVIDENCE — infraestructura automatizada retirada: - total 1285 PASS.

La cantidad puede crecer. El requisito es que la suite vigente sea descubierta y pase, no forzar siempre la cifra histórica.

33.3. Primera validación

1. abrir Test Runner;
2. ejecutar EditMode completo;
3. guardar reporte o captura;
4. ejecutar PlayMode completo;
5. esperar restauración de escenas;
6. revisar Console;
7. comprobar Git.

33.4. Fallos

- no pulsar Run repetidamente sin analizar;
- comprobar orden y contaminación de estado;
- comprobar assets generados;
- comprobar constraints de tests;
- comprobar escenas abiertas;
- registrar defecto si es producto;
- reparar test si la regla vigente cambió formalmente.

34. Validación de escenas y catálogos

34.1. Smoke tests

Después de una restauración:

- Bootstrap abre MainMenu;
- MainMenu muestra slots;
- Store funcional sigue disponible durante migración;
- StoreInitial abre sin referencias rotas cuando se valide;
- TestLab ejecuta fixtures aislados;
- registry carga catálogos;
- no hay managers duplicados.

34.2. Missing scripts

Buscar:

- Missing (Mono Script);
- prefabs con referencias vacías;
- escenas con GUID no resuelto;
- materiales rosas;
- renderers sin mesh;
- catálogos con entradas nulas.

No guardar la escena hasta comprender la causa, porque Unity puede serializar pérdida de referencias.

35. Validación de build mínima

La guía de build detallada será `11_Build_y_Versioning_Guide.md`. Para validar setup:

1. activar `Windows_Development`;
2. comprobar escenas;
3. generar en carpeta limpia fuera de Assets;
4. ejecutar EXE;
5. alcanzar MainMenu;
6. entrar en la escena configurada;
7. cerrar correctamente;
8. revisar `Player.log`;
9. comprobar Git limpio;
10. registrar versión y SHA.

Una build exitosa no valida por sí sola el Golden Path.

36. Player.log y diagnóstico

36.1. Player Log

Está activado y es obligatorio para builds internas.

Revisar:

- excepciones;
- errores repetidos;
- missing references;
- shader fallbacks;
- fallos de save;
- escenas ausentes;
- paquetes o plugins;
- warnings de rendimiento relevantes.

36.2. Evidencia

Archivar el log con:

- Build ID;
- versión;
- SHA;
- fecha;
- recorrido;
- resultado.

No publicar rutas personales sin sanitizar.

37. Limpieza segura

37.1. Qué puede regenerarse

Con Unity cerrado y Git limpio, pueden regenerarse:

- Library;
- Temp;
- Obj;
- archivos IDE;
- algunos caches locales.

37.2. Qué no debe borrarse

- Assets;
- .meta;
- Packages/manifest.json;
- Packages/packages-lock.json;
- ProjectSettings;
- Build Profiles;
- catálogos;
- escenas;
- documentación de evidencia.

37.3. Orden de recuperación

1. registrar error;
2. cerrar Unity;
3. guardar estado Git;
4. comprobar espacio/permisos;
5. retirar solo cache local necesaria;
6. abrir con Editor correcto;
7. esperar importación;
8. ejecutar tests;
9. comparar Git.

“Borrar Library” es una herramienta de diagnóstico de caché, no una solución universal.

38. Problemas habituales

38.1. Paquetes no resueltos

- confirmar conexión;
- revisar manifest/lock;
- comprobar Editor correcto;
- no borrar lockfile primero;
- revisar Package Manager log;
- comparar con baseline.

38.2. Materiales rosas

- comprobar pipeline;
- shader;
- renderer asset;
- importación;
- platform override;
- no convertir masivamente sin backup.

38.3. Scripts no compilan

- primer error;
- asmdef;
- namespace;
- paquete;
- define constraint;
- Editor-only API;
- archivo duplicado.

38.4. Tests no aparecen

- asmdef de test;
- reference a Test Runner;
- include platform;
- UNITY_INCLUDE_TESTS;
- errores de compilación;
- ubicación del script.

38.5. Build incluye tests

Verificar define constraints y referencias. No resolver eliminando tests.

38.6. StoreInitial muestra doble tienda

- revisar flag procedural;
- StoreRuntimeSettings;
- composition root;
- environment prefab;
- instancias duplicadas;
- escena y fallback.

38.7. UI mueve al jugador

- EventSystem;
- IsPointerOverGameObject;
- input context;
- orden de procesamiento;
- clic residual al cerrar panel.

38.8. Git muestra cientos de assets

- Editor incorrecto;
- herramienta masiva;
- reserialización;
- .gitattributes;
- import settings;
- cambios de pipeline.

No commit hasta clasificar.

39. Incorporación de una máquina nueva

39.1. Checklist previo

- ☐ Windows actualizado y estable.
- ☐ Drivers registrados.
- ☐ Unity Hub instalado.
- ☐ Editor 6000.3.18f1 instalado.
- ☐ Windows Build Support instalado.
- ☐ IDE y workload instalados.
- ☐ Git configurado.
- ☐ Acceso al repositorio confirmado.
- ☐ Espacio suficiente.

39.2. Checklist de clonación

- ☐ Clonado completo.
- ☐ Rama y SHA correctos.
- ☐ Working copy limpia.
- ☐ Assets, Packages, ProjectSettings presentes.
- ☐ .gitignore y .gitattributes presentes.
- ☐ No hay archivos faltantes.

39.3. Checklist de Unity

- ☐ Abre con Editor exacto.
- ☐ Paquetes resueltos.
- ☐ Cero errores rojos.
- ☐ Assemblies descubiertos.
- ☐ URP activo.
- ☐ Input actions disponibles.
- ☐ Runtime registry válido.
- ☐ Escenas presentes.
- ☐ EditMode pasa.
- ☐ PlayMode pasa.
- ☐ Build externa arranca.
- ☐ Player.log revisado.
- ☐ Git permanece limpio.

39.4. Registro

Crear o actualizar un Environment Record sin datos sensibles.

40. Checklist antes de un sprint

- ☐ Pull/fetch completado.
 - ☐ SHA registrado.
 - ☐ Working copy limpia.
 - ☐ Editor correcto.
 - ☐ Paquetes sin cambios no aprobados.
 - ☐ Console limpia.
 - ☐ Tests base pasan.
 - ☐ Build Profile correcto.
 - ☐ escena objetivo abre.
 - ☐ catálogos validados.
 - ☐ espacio libre.
 - ☐ backup/evidencia preparada.
 - ☐ alcance y exclusiones del sprint leídos.
-

41. Añadir un paquete

41.1. Definition of Ready

Antes de añadir:

- problema demostrado;
- alternativa sin paquete evaluada;
- versión Released;
- compatibilidad con Unity;
- licencia;
- mantenimiento;
- impacto de build;
- impacto de assemblies;
- plan de prueba;
- plan de retirada.

41.2. Procedimiento

1. working copy limpia;
2. rama/cambio aislado;
3. añadir desde Package Manager;
4. revisar manifest y lock;
5. esperar importación;
6. revisar dependencias transitivas;
7. compilar;
8. ejecutar suites;
9. generar build;
10. registrar decisión.

41.3. Rechazo

No añadir por:

- una única función trivial;

- tutorial desactualizado;
 - “por si acaso”;
 - plantilla;
 - reducir unas pocas líneas a costa de dependencia permanente.
-

42. Actualizar un paquete

42.1. Un paquete cada vez

Evita mezclar causas de regresión.

42.2. Evidencia

- versión anterior/nueva;
- changelog;
- APIs afectadas;
- diff del manifest/lock;
- tests;
- build;
- Player.log;
- rendimiento si aplica;
- rollback.

42.3. URP/Input/Navigation

Estos paquetes afectan assets serializados y comportamiento. Sus upgrades requieren especial atención a escenas, materiales, renderer assets, acciones, NavMesh y tests.

43. Actualizar Unity

43.1. REQUIERE ADR

Cambiar Unity requiere:

- motivación;
- versión objetivo LTS;
- compatibilidad de paquetes;
- copia/branch de upgrade;
- clone limpio;
- backup;
- reimportación;
- diff de ProjectSettings;
- suites completas;
- Golden Path;
- build;
- profiling;
- actualización documental.

43.2. Prohibiciones

- no abrir la única working copy con una versión superior;
- no aceptar resave masivo sin revisión;
- no mezclar upgrade con features;
- no borrar la baseline anterior antes de aprobar.

43.3. Rollback

Debe existir una ruta para volver a la versión y commit anteriores sin assets parcialmente migrados.

44. Cambios que requieren ADR

- versión de Unity;
- render pipeline;
- plataforma objetivo;
- backend de scripting de producción;
- sistema de input;
- estrategia de Addressables;
- Steamworks;
- estructura principal de assemblies;
- ownership de escenas;
- sustitución de ApplicationRoot;
- estrategia de save;
- cambio de serialization mode;
- desactivar Visible Meta Files;
- introducir un DI container global;
- eliminar Store fallback antes del gate;
- cambiar estrategia de asset identity;
- introducir servicios online.

Un cambio menor de calidad o importación puede no necesitar ADR, pero sí evidencia si afecta al gate.

45. Configuración que no debe cambiarse silenciosamente

Área	Configuración protegida
Editor	6000.3.18f1
Serialization	Force Text
Version Control	Visible Meta Files
Color	Linear
Packages	versiones directas aprobadas
Scenes	Bootstrap primero
Assemblies	límites Domain/Application
Input	Input System y contextos exclusivos
Registry	referencias preloaded
Save	rutas y schema versionado
Build	Windows x64 y profile identificado
Identity	Company/Product/Identifier

46. Deuda y discrepancias actuales

46.1. Tabla de deuda

ID	Observación	Riesgo
SETUP-DEBT-001	Build Profile todavía usa Store	StoreInitial no está en build
SETUP-DEBT-002	StoreRuntimeSettings apunta a Store y blockout activo	doble shell o escena equivoco
SETUP-DEBT-003	resolución por defecto 1024×768	no representa objetivo de va
SETUP-DEBT-004	ventana no redimensionable	posible fricción PC
SETUP-DEBT-005	Localization implementada con tablas y assets validados	ES/EN no queda demostrado
SETUP-DEBT-006	no hay tags/layers propios	no es defecto; limita máscar
SETUP-DEBT-007	no hay .editorconfig ni analyzers	estilo no automatizado
SETUP-DEBT-008	no hay Git LFS	crecimiento de binarios
SETUP-DEBT-009	asset de Input incluye acciones genéricas	contrato confuso
SETUP-DEBT-010	TestLab incluido en Development profile	no apto para release

46.2. Regla

Una deuda no se “arregla” modificando archivos sin criterio de aceptación. Cada acción debe indicar responsable, prueba, build y rollback.

47. Definition of Done del setup

Una restauración o configuración de máquina está cerrada cuando:

1. usa Unity exacto;
2. tiene módulos requeridos;
3. clona la baseline completa;
4. Package Manager resuelve manifest y lock;
5. Console no tiene errores rojos;
6. Force Text y Visible Meta Files se conservan;
7. URP PC está activo;
8. assemblies compilan sin ciclos;
9. Domain/Application no tienen Engine References;
10. Bootstrap es primera escena del profile activo;
11. registry e input preloaded se resuelven;
12. escenas abren sin missing scripts;
13. EditMode pasa;
14. PlayMode pasa;
15. build Windows x64 arranca;
16. Player.log está revisado;
17. Git queda limpio;
18. Environment Record está actualizado;
19. diferencias aceptadas están registradas;
20. no se han introducido secretos.

48. Criterios de aceptación

ID	Criterio	Evidencia
SETUP-001	ProjectVersion.txt declara 6000.3.18f1.	archivo y About Unity
SETUP-002	Windows Build Support está disponible.	build externa
SETUP-003	manifest.json contiene las siete dependencias directas aprobadas.	diff/revisión
SETUP-004	packages-lock.json resuelve sin Preview.	Package Manager

ID	Criterio	Evidencia
SETUP-005	Company, Product, Identifier y Version son correctos.	Player Settings
SETUP-006	Color Space es Linear.	Project Settings
SETUP-007	Visible Meta Files está activo.	Version Control Settings
SETUP-008	Force Text está activo.	Editor Settings
SETUP-009	Git ignora carpetas generadas.	status limpio
SETUP-010	.gitattributes normaliza texto y clasifica binarios.	revisión
SETUP-011	los doce asmdefs se descubren.	Inspector/compilación
SETUP-012	Domain y Application usan No Engine References.	asmdefs
SETUP-013	Bootstrap es primera escena del profile.	Build Profile
SETUP-014	StoreInitial conserva GUID propio.	.meta
SETUP-015	RuntimeAssetRegistry y Input Actions se resuelven.	arranque/tests
SETUP-016	PC_RPAsset está asignado a Standalone.	Graphics/Quality
SETUP-017	Input System funciona sin propagación UI→mundo.	PlayMode/manual
SETUP-018	AI Navigation alcanza puntos críticos.	recorrido
SETUP-019	no hay missing scripts ni materiales rosas.	revisión escena
SETUP-020	EditMode vigente pasa.	reporte
SETUP-021	PlayMode vigente pasa.	reporte
SETUP-022	EXE Windows x64 arranca desde Bootstrap.	build record
SETUP-023	Player.log no tiene errores bloqueantes.	log
SETUP-024	Git permanece limpio tras abrir, probar y cerrar.	status
SETUP-025	el registro de entorno no contiene secretos.	revisión
SETUP-026	cambios de Unity/paquetes disponen de ADR o registro.	documentación
SETUP-027	Store no se sustituye antes de validar StoreInitial.	gate Sprint 16
SETUP-028	Localization ES/EN se valida antes de H6.	QA matrix
SETUP-029	resolución objetivo y ventana se deciden antes de candidato.	Player Settings/build
SETUP-030	la guía y baseline reflejan la configuración real.	trazabilidad

49. Mantenimiento

Actualizar esta guía cuando cambie:

- Editor;
- módulos;
- paquetes;
- ProjectSettings protegido;
- estructura principal;
- assemblies;
- escenas o Build Profiles;
- pipeline;
- input;
- navegación;
- herramientas de importación;
- proceso de apertura;
- criterios de QA.

No reescribir el histórico. Registrar qué decisión sustituye a la anterior.

El archivo debe guardarse como:

Documentacion/
└─ 10_Unity_Project_Setup_Guide.md

50. Mapa de configuración serializada

50.1. ProjectVersion

ProjectVersion.txt es el primer control de compatibilidad. Unity Hub puede mostrar varias instalaciones con nombres similares; el archivo del proyecto es la referencia. La revisión completa evita que dos builds con el mismo número visible pero distinta revisión se traten como idénticas.

Comprobar siempre:

- m_EditorVersion;
- m_EditorVersionWithRevision;
- instalación seleccionada en Hub;
- versión registrada en build y evidencia.

50.2. ProjectSettings.asset

Este archivo contiene identidad, plataforma, resolución, logging, input y opciones de Player. Debe revisarse por bloques, no como un diff opaco.

Bloque	Campos de revisión	Riesgo de cambio
Identity	Company, Product, Identifier, Version	saves, distribución y trazabilidad
Display	resolución, fullscreen, resizable, background	UX y compatibilidad PC
Logging	Player Log	diagnóstico de QA
Rendering	color space, graphics APIs, batching	visual y rendimiento
Scripting	backend, unsafe, deterministic compilation	compatibilidad y build
Input	active input handler	pérdida total de controles
Preloaded	Input Actions y Runtime Registry	bootstrap y datos

Un cambio de Player Settings debe registrar una captura o diff legible. No debe mezclarse con una importación masiva de assets, porque dificultaría detectar la causa de una regresión.

50.3. EditorSettings.asset

Elementos protegidos:

- serialización textual;
- generación de proyectos;
- opciones de Play Mode;
- texture streaming en Editor y Play Mode;
- asset pipeline;
- cache server;
- prefab auto-save.

La baseline tiene Enter Play Mode Options activado con máscara serializada 0. No se debe interpretar el número sin comprobar el comportamiento de domain reload y scene reload en el Editor usado. Cualquier optimización de entrada a Play Mode debe probar:

- estáticos;
- singletons;
- event subscriptions;
- ApplicationRoot;
- tests ejecutados en orden distinto;
- salida y reentrada en Play Mode.

50.4. GraphicsSettings y URP Global Settings

Revisar:

- pipeline global;
- shader stripping;
- renderer features;
- render graph o compatibilidad si cambia;
- luces lineales;
- volumen global;
- recursos de URP.

Una referencia de pipeline perdida puede convertir materiales en rosa y alterar todas las escenas. No debe repararse asignando un asset arbitrario con nombre parecido; se verifica GUID y profile de calidad.

50.5. QualitySettings

Cada profile tiene una referencia de pipeline. Al añadir un profile:

1. copiar solo como punto de partida;
2. asignar nombre y plataformas;
3. revisar pipeline asset;
4. revisar sombras, LOD, texturas y VSync;
5. probar cambio en runtime si se expone al usuario;
6. guardar métricas por profile;
7. actualizar QA Matrix.

50.6. TagManager

El archivo serializa índices. Cambiar el nombre de una layer no actualiza necesariamente todas las expectativas humanas o herramientas externas. Reservar una tabla de ownership si se crean layers propias.

50.7. TimeManager y Physics

Los valores se consideran parte de la simulación. Una modificación puede cambiar:

- velocidad aparente;
- resolución de colisiones;
- navegación;
- animación;
- reproducibilidad de tests;
- coste de CPU.

No cambiar timestep para “hacer que vaya más fluido”. Primero perfilar la causa.

50.8. AudioManager

Cambios de voces, buffer o sample rate pueden resolver un problema en una máquina y crear latencia o cortes en otra. Toda modificación requiere:

- hardware de prueba;
- escena con carga representativa;
- registro de latencia/cortes;
- comparación antes/después;
- build externa.

50.9. EditorBuildSettings y Build Profiles

La lista global y la lista del profile pueden diferir. En Unity 6, el profile puede sobrescribir la lista global. QA debe registrar qué profile estaba activo. Nunca asumir que modificar EditorBuildSettings.asset basta si el profile usa `m_OverrideGlobalSceneList`.

51. Perfiles detallados de importación

51.1. Arquitectura

Para muros, suelo, marcos y elementos fijos:

- escala métrica verificada;
- pivote coherente con autoría;
- materiales URP;
- lightmap UV cuando se use baking;
- static flags según función;
- collider técnico separado cuando la malla visual sea compleja;
- sin scripts de gameplay incrustados en el FBX;
- prefab bajo Prefabs/Architecture;
- instancia bajo StoreInitialEnvironment/Architecture.

Criterio de aceptación: visual, collider, grid y navegación describen el mismo espacio.

51.2. Mobiliario colocable

- pivote en base y punto de colocación;
- footprint definido por datos, no por bounds cambiantes;
- orientación inicial conocida;
- LODGroup si aporta valor;
- colliders simples;
- slots o anchors explícitos;
- prefab separado del FBX;
- definición ScriptableObject con ID estable;
- miniatura/icono si aparece en UI;
- material compartido cuando corresponde.

Probar rotaciones 0°, 90°, 180° y 270°. Un mueble visualmente simétrico puede tener footprint o puntos de interacción asimétricos.

51.3. Displays

Además de mobiliario:

- capacidad lógica;
- slots visuales;
- productos compatibles;
- origen de reposición;
- punto de interacción;
- representación de vacío/parcial/lleño;
- persistencia de asignación.

No usar el número de hijos visuales como capacidad autoritativa.

51.4. Productos y packaging

- escala consistente entre familias;

- orientación frontal;
- punto de apoyo;
- materiales legibles;
- LOD si el volumen lo requiere;
- prefab registrado en catálogo;
- ID de producto independiente del nombre del prefab;
- representación de caja separada cuando el flujo la necesite.

Una carátula o etiqueta no debe incluir marcas reales sin autorización.

51.5. Personajes

- rig y avatar;
- escala;
- clips requeridos;
- materiales;
- LOD;
- collider/cápsula;
- punto de navegación;
- perfil lógico separado;
- variantes visuales sin cambiar identidad de dominio;
- animator controller compatible.

Validar con varios agentes simultáneos, no solo un personaje aislado.

51.6. Texturas de entorno

- resolución proporcional a tamaño en pantalla;
- tiling;
- normal map;
- mask maps si el shader las utiliza;
- compresión de Standalone;
- mipmaps;
- anisotropic filtering para superficies oblicuas;
- ausencia de seams críticos.

51.7. UI

- Sprite (2D and UI);
- alpha correcto;
- compresión que no introduzca halos;
- nine-slice cuando proceda;
- tamaño y PPU coherentes;
- atlas evaluado por lote;
- variantes de escala;
- contraste y legibilidad.

51.8. Audio por categoría

Categoría	Importación inicial	Validación
UI	clip corto, latencia baja	repetición y volumen
SFX	según duración y concurrencia	clipping y voice count
Ambience	loop estable	transición y memoria
Music	streaming tras medir	seek, loop y carga
Voz	compresión orientada a claridad	subtítulos y localización

51.9. Animaciones

- separar clips correctamente;
- revisar loop pose;
- decidir root motion;
- retirar curvas no usadas si aporta ahorro;
- no depender de nombres de clips generados por DCC;
- mantener una tabla de eventos permitidos;
- validar transición en Animator.

51.10. Presets

Los Presets de importación pueden reducir errores, pero no deben aplicarse a todos los assets por extensión. Crear presets por categoría y documentar su filtro. Cambiar un preset puede reimportar cientos de assets; se trata como cambio de alto riesgo.

52. Matriz de validación por tipo de cambio

Cambio	Compilación	EditMode	PlayMode	Escena manual	Build
C# Domain	sí	completa	afectada	regresión	según sistema
C# Presentation	sí	afectada	completa	UI/escena	sí
asmdef	sí	completa	completa	smoke	sí
paquete	sí	completa	completa	Golden Path	sí
ProjectSettings	sí	afectada	afectada	sí	sí
Build Profile	sí	smoke	smoke	flujo de escenas	obligatorio
escena	sí	autoría	completa	relevante	obligatorio
prefab crítico	sí	autoría	integración	obligatorio	gato
catálogo	sí	validación	integración	flujo afectado	gato
textura/material	no siempre	authoring	visual	obligatorio	recomendado
FBX/LOD	no siempre	authoring	visual/nav	obligatorio	recomendado
audio	no siempre	catálogo	audio	obligatorio	recomendado
localización	sí si código	claves	UI	ES/EN	sí
Unity upgrade	completa	completa	completa	Golden Path	obligatorio

52.1. Cambio pequeño no significa riesgo pequeño

Un cambio de una línea en el Build Profile puede impedir arrancar. Un rename de asset puede romper referencias si pierde su .meta. La profundidad de QA se decide por impacto, no por número de archivos.

52.2. Validación de autoría

Las herramientas deben poder detectar:

- referencias nulas;
- IDs duplicados;
- prefabs sin componentes;
- catálogos inconsistentes;
- LODGroup incompleto;
- materiales incompatibles;
- colliders ausentes;
- escenas fuera de profile;
- assets técnicos en rutas representativas.

52.3. Validación posterior a merge

Aunque una rama pasara tests, después de merge se vuelve a comprobar:

- resolución de conflictos;
 - paquete lock;
 - escenas/prefabs;
 - suites afectadas;
 - build de gate;
 - Git limpio.
-

53. Recuperación ante daños o pérdida de referencias

53.1. Principio

La recuperación conserva evidencia antes de intentar reparar. No sobrescribir el único estado que muestra el problema.

53.2. Escena dañada

1. cerrar Unity sin guardar si la pérdida apareció al abrir;
2. copiar la escena y .meta fuera de la working copy para análisis;
3. revisar Git diff;
4. identificar GUID o script ausente;
5. restaurar desde commit si procede;
6. reabrir con Editor correcto;
7. validar referencias;
8. ejecutar tests y recorrido;
9. documentar causa.

No copiar únicamente la escena de otra rama sin sus prefabs y datos relacionados.

53.3. Prefab con Missing Script

- localizar GUID del script en YAML;
- comprobar .meta del script;
- comprobar assembly y namespace;
- comprobar si el tipo fue renombrado;
- restaurar o migrar explícitamente;
- no pulsar Remove Component en masa.

53.4. Material perdido

- localizar GUID;
- comprobar material y .meta;
- comprobar shader;
- comprobar pipeline;
- revisar cambios de importación;
- restaurar referencia, no crear clones con nombres similares.

53.5. Catálogo roto

- no iniciar una partida que pueda persistir estado parcial;
- ejecutar validador;
- comparar IDs;
- restaurar asset;

- comprobar registry;
- ejecutar tests de datos y save/load.

53.6. Manifest dañado

- restaurar desde Git;
- no reconstruir versiones “de memoria”;
- conservar lockfile correspondiente;
- abrir y esperar resolución;
- comprobar que no aparecen paquetes Preview o de plantilla.

53.7. Library corrupta

Síntomas posibles:

- imports incoherentes;
- cache de shader;
- referencias que vuelven tras reinicio;
- errores internos de importer.

Procedimiento:

1. confirmar que archivos versionados están sanos;
2. cerrar Unity/Hub;
3. renombrar Library en lugar de borrar si se necesita análisis;
4. abrir y reimportar;
5. comparar resultado;
6. eliminar backup local solo después de validar.

53.8. Conflicto de escena o prefab

Si el conflicto no es trivial:

- no editar a ciegas marcadores de merge;
 - identificar objetos y fileIDs;
 - considerar rehacer una parte en Unity;
 - comparar ambos cambios;
 - validar escena completa;
 - documentar trabajo descartado.
-

54. Seguridad y secretos

54.1. Archivos prohibidos en Git

- tokens de Steam o servicios;
- contraseñas;
- certificados privados;
- keystores;
- .env reales;
- logs con datos personales;
- dumps de memoria sin revisar;
- configuraciones de máquina con usuario/rutas privadas.

54.2. Variables y configuración local

Cuando se introduzcan servicios externos:

- plantilla sin secretos;
- almacenamiento local seguro;
- inyección en build/CI;
- rotación;
- permisos mínimos;
- documentación de recuperación.

54.3. Assets de terceros

Registrar:

- origen;
- licencia;
- versión;
- modificaciones;
- restricciones de redistribución;
- créditos.

No incluir packages o samples de terceros sin verificar qué archivos pueden versionarse.

54.4. Logs y builds

Antes de compartir:

- revisar rutas;
 - revisar IDs personales;
 - eliminar tokens;
 - comprobar save files;
 - etiquetar build y propósito;
 - no publicar una Development Build abierta sin necesidad.
-

55. Campaña de aceptación de una instalación nueva

55.1. Fase A — Integridad estática

- estructura raíz;
- ProjectVersion;
- manifest/lock;
- hashes o Git;
- ProjectSettings;
- asmdefs;
- escenas y metas;
- registry;
- input actions.

55.2. Fase B — Importación

- paquetes resueltos;
- Console;
- shaders;
- materiales;
- prefabs;
- catálogos;
- no reserialización inesperada.

55.3. Fase C — Compilación y tests

- compilación limpia;
- EditMode;
- PlayMode;
- Input System tests;
- authoring validators;
- repetición para detectar contaminación.

55.4. Fase D — Escenas

- Bootstrap;
- MainMenu;
- Store durante migración;
- StoreInitial cuando proceda;
- TestLab;
- transiciones;
- ApplicationRoot único.

55.5. Fase E — Build

- profile correcto;
- carpeta limpia;
- EXE externa;
- MainMenu;
- escena de tienda;
- salida;
- Player.log.

55.6. Fase F — Repositorio

- cerrar Unity;
- revisar Git;
- clasificar cambios;
- restaurar cualquier archivo generado no deseado;
- registrar resultado.

55.7. Resultado

La campaña termina como:

- PASS;
- PASS WITH OBSERVATION;
- FAIL;
- BLOCKED.

Un PASS WITH OBSERVATION debe incluir una observación no bloqueante y su seguimiento. No usarlo para ocultar errores rojos, tests fallidos o una build que no arranca.

56. Handoff del entorno

Un handoff de setup debe indicar:

- Editor y revisión;
- plataforma y módulos;
- SHA;

- rama;
- versión de aplicación;
- package diff;
- ProjectSettings modificados;
- profile activo;
- suites y resultados;
- build y log;
- deuda o bloqueos;
- siguiente acción;
- acciones prohibidas.

Debe poder comprenderse sin depender de mensajes de chat o memoria personal.

Anexo A. Comandos de diagnóstico

A.1. Git

```
git status
git rev-parse HEAD
git diff -- ProjectSettings Packages Assets/_Project/Settings
```

A.2. Comprobación de archivos

```
Assets/
Packages/manifest.json
Packages/packages-lock.json
ProjectSettings/ProjectVersion.txt
ProjectSettings/ProjectSettings.asset
```

Los comandos son ejemplos. No deben ejecutarse sobre una working copy con cambios no comprendidos.

Anexo D. Resumen de no negociables

1. Editor exacto.
2. Manifest y lock versionados.
3. Force Text.
4. Visible Meta Files.
5. Bootstrap primero.
6. No Engine References en Domain/Application.
7. GUIDs estables.
8. StoreInitial no sustituye Store antes del gate.
9. Tests y build después de cambios estructurales.
10. Player.log y Git limpio como evidencia.

Estado del documento: fuente vigente de configuración, restauración y validación inicial del proyecto Unity para la nueva carpeta Documentacion/.

Actualización Unity W0 - cámara y pivotes

Estado operativo vigente tras W0

Elemento	Estado vigente
Baseline de entrada	main@efbc1a885a1d6819f764ec8cff8a465ad1986661 / aplicación 0.0.26
W0 documental	COMPLETED / DOCUMENTAL PASS
Decisiones funcionales abiertas	0
Sprint17_Phase1	APPROVED / IMPLEMENTED / VALIDATED / CLOSED
H6	APPROVED / IMPLEMENTED / VALIDATED / CLOSED
Vertical Slice	ACCEPTED / IMPLEMENTED / VALIDATED / CLOSED

Wall occlusion

- La policy efectiva de H6 es false y tiene prioridad sobre PlayerPrefs, assets o valores serializados antiguos.
- El control de usuario queda oculto durante H6. Una preferencia antigua se migra o ignora, pero no puede reactivar la función.
- La regresión debe comprobar cámara orbital, zoom, paredes, entrada, checkout, displays y almacén en Editor y build externa.

Pivotes y GroundAnchor

- El contrato de colocación es base-centro en espacio local.
- Un FBX con pivote incompatible se envuelve en un prefab raíz o expone GroundAnchor; no se compensa repetidamente desde runtime.
- La migración de instancias existentes registra versión/marcador y se ejecuta como máximo una vez.
- Escenas y saves de 0.0.26 deben conservar posición, rotación, footprint, colliders y navegación.
- Tras W7, el contrato queda congelado y cualquier excepción requiere ADR/cambio trazado.

Regla de cierre y no propagación

- W0–W8 están completadas, validadas y cerradas en la baseline 0.0.26.
- Sprint17_Phase1 y W8 están completadas, validadas y cerradas.
- H6 y la Vertical Slice disponen de aceptación propia y están cerrados sobre 0.0.26.
- La Vertical Slice queda cerrada; los sistemas Post-H6 se gestionarán como evolución posterior mediante control de cambios.
-